# GLB: Lifeline-based Global Load Balancing library in X10

Wei Zhang, Olivier Tardieu, David Grove, Benjamin Herta, Tomio Kamada (Kobe U/RIKEN)
Vijay Saraswat, Mikio Takeuchi

(Besides Tomio Kamada, the other authors are at IBM Research)

December 9, 2013

We present GLB, a programming model and an associated implementation that can handle a wide range of irregular parallel programming problems running over large-scale distributed systems. GLB is applicable both to problems that are easily load-balanced via static scheduling and to problems that are hard to statically load balance. GLB hides the intricate synchronizations (e.g., inter-node communication, initialization and startup, load balancing, termination and result collection) from the users. GLB internally uses a version of the lifeline graph based work-stealing algorithm proposed by Saraswat et al [23]. Users of GLB are simply required to write several pieces of **sequential code** that comply with the GLB interface. GLB then schedules and orchestrates the parallel execution of the code correctly and efficiently at scale.

We have applied GLB to two representative benchmarks: Betweenness Centrality (BC) and Unbalanced Tree Search (UTS). Among them, BC can be statically load-balanced whereas UTS cannot. In either case, GLB scales well – achieving nearly linear speedup on different computer architectures (Power, Blue Gene/Q, and K) – up to 16K cores.

## 1 Introduction

### 1.1 Why do we need GLB

Parallel programming is significantly more challenging than sequential programming. Given the same input, a sequential program always produces the same result. That is not true for parallel programs. Programmers need to ensure that given the same input, a parallel program can produce the same result under any possible interleaving. In addition, programmers also need to design intricate synchronization schemes to balance the workload among computing resources so that the deployed program can achieve good performance.

Facing these challenges, many programmers desire a programming model that can can hide the synchronization details from them. Many such programming models have been proposed. Among them, MapReduce [7] is one of the most widely used. Programmers only need to provide the sequential mapper and reducer function and the MapReduce framework takes care of input partition, scheduling, inter-machine communication and fault tolerance. While effective for many problems, MapReduce is not applicable to highly-irregular workload. One heavily loaded mapper or reducer can severely downgrade the whole system's performance. Work-stealing [9] is among the first works that handle the irregular workload on shared-memory machine. However, many such techniques are not directly applicable to distributed-memory machines, which are the de-facto programming environment for scale-out computing.

We propose GLB, a Global Load Balancing framework, based on lifeline graph work-stealing algorithm [23]. GLB can handle highly irregular problems, where workload on each computing node can be widely different and unpredictable. GLB works on distributed-memory system and can deliver linear speedup and perfect scaling up to 16K cores. In addition, GLB provides a number of parameters for users to tune. Programmers can control the number of random/lifeline victims and task granularity to increase computation throughput or decrease work-stealing response latency, without knowing the underlying complicated synchronization scheme.

### 1.2 Why do we choose X10

X10 is a high-performance, high-productivity programming language for scale out computing being developed at IBM. It is a class-based, strongly-typed, garbage-collected, object-oriented language [22, 21]. To support concurrency and distribution, X10 uses the Asynchronous Partitioned Global Address Space programming model (APGAS [20]). This model introduces two key concepts – places and asynchronous tasks – and a few mechanisms for coordination (`finish`, `async`, `at`, `atomic`). With these, APGAS can express both regular and irregular parallelism, message-passing-style and active-message-style computations, fork-join and bulk-synchronous parallelism. In contrast to hybrid models like MPI+OpenMP, the same constructs underpin both intra- and inter-place concurrency.

The X10 language is implemented via source-to-source compilation to either Java (*Managed X10*) or C++ (*Native X10*) and is available on a wide range of operating systems and hardware platforms ranging from laptops, to commodity clusters, to supercomputers.

Using X10 to implement GLB simplified both the internal implementation of the library and its end-user API. X10's high-level support for distribution and concurrency allowed a concise and efficient specification of the library and enabled rapid prototyping and experimentation with design choices. The X10 language's intrinsic support for distributed computing, specifically for data serialization between nodes, simplifies the end-user programming task. Users can simply specify sequential, single-place data structures and the GLB implementation can rely on X10 language support to efficiently transmit user-defined data types between across nodes without requiring any explicit data serialization code to be written by the user.

## 2 Design principle

### 2.1 What type of problems are GLB applicable to

GLB is applicable to problems described as following. There is an initial bag (multiset) of *tasks* (may be of size one). A task usually has a small amount of associated state, but is permitted to access (immutable) "reference state" that can be replicated across all places. Consequently, tasks are *relocatable*: they can be executed at any place.

Tasks can be executed. Executing a task may only involve local, self-contained work (no communication). During execution, a task is permitted to create zero or more tasks, and produce a result of a given (pre-specified) type `Z`. The user is required to specify a reduction operator of type `Z`.

The GLB framework is responsible for distributing the generated collection of tasks across all nodes. Once no more tasks are left to be executed, the single value of type `Z` (obtained by reducing the results produced by all tasks) is returned.

Since the execution of each task depends only on the state of the task (and on immutable external state), the set of results produced on execution will be the same from one run to another, regardless of how the tasks are distributed across places. Since the user supplied reduction operator is assumed to be associative and commutative, the result of execution of the problem is determinate. Thus GLB is a determinate application library that requires the user to provide a few sequential pieces of code and handles concurrency, distribution and communication automatically.

The GLB framework is applicable to a wide variety of tasks. For a simple example, consider the problem of computing the $n$'th Fibonacci number in the naive recursive way. Here the state of the task can be summarized by just one number (long), i.e. $n$. Execution of the task yields either a result (for $n < 2$), or two tasks (for $n - 1$ and $n - 2$). The results from all tasks need to be reduced through arithmetic addition.

All *state space search algorithms* from AI fall in the GLB problem domain. Such algorithms are characterized by a space of states, an initial state, and a generator function which given a state generates zero or more states. The task is to enumerate all states (perhaps with a cutoff), apply some function, and combine the results. An example of such an application is the famous $N$-Queens problem.

### 2.2 Execution overview

Figure 1 shows the overall flow of GLB. For each GLB program, users need to provide two pieces of sequential code *TaskQueue* and *TaskBag*. In addition, users can also provide an *Initialization* method that tells GLB how to initialize the workload at a root place if the workload cannot be statically scheduled among all places.

*TaskQueue* embodies the sequential computation for this problem and a result reduction function. *TaskBag* embodies the task container's data structure and its split/merge method.

Given the *TaskQueue* and *TaskBag* implementation, GLB initializes the workload for each *Worker*. Workers are GLB internal computing/load-balancing engines and they are transparent to GLB users. *Workers* process task items by call the `process` method provided by *TaskQueue*. *Workers* balance workload using the lifeline based work-stealing algorithm. GLB splits *TaskBag*s from victim *Worker* and merges them to thief *Worker*

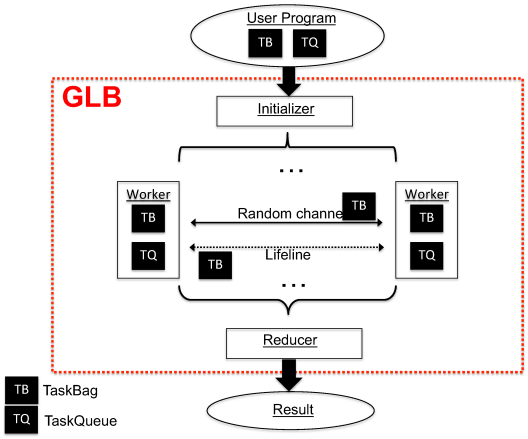


Figure 1: GLB overview

by calling the `split/merge` methods provided by *TaskBag*. Once all workers finish working, GLB invokes the `reducer` function, provided by *TaskQueue*, to return the final result.

In the following sections, Section 2.3 discusses what are required from users, Section 2.4 discusses the GLB internals, Section 2.5 discusses how to apply GLB to the UTS problem, Section 2.6 discusses how to apply GLB to the BC problem.

### 2.3 What do users need to provide

**TaskQueue** Users need to provide an implementation of following methods:

1. `process(n:Long):Boolean` This method describes the sequential computation of the problem. It iterates over $n$ items in its TaskBag and computes each one of them. It processes $n$ items if available and returns true; otherwise it processes all available ($< n$) items and returns false. When GLB sees a Worker's process(n) returns false, it will schedule this Worker to steal from others.

2. `split():TaskBag` Split its TaskBag and returns the split half. This method returns null if the TaskBag is too small to split.

3. `merge(TaskBag)` Merge the incoming TaskBag to its own TaskBag. Both split and merge method functions are wrapper functions that will call TaskBag's split/merge functions.

4. `getResult()` Returns local result.

5. `reduce()` Result reduction function. Applying this function to each Worker's result yields the final result.

**TaskBag** Users need to provide a custom class extending *TaskBag* and appropriately implementing `split` and `merge` methods. They provide the functionalities that *TaskQueue*'s `split/merge` methods need. GLB provides a default ArrayList-based TaskBag implementation. In this default implementation, split method is implemented by removing half of the elements from the end of the ArrayList and returning this removed half ArrayList; merge method is implemented by adding all the elements from the incoming ArrayList to the local ArrayList.

Once the customized TaskQueue and TaskBag are implemented, it is fairly easy for users to invoke GLB library by calling

GLB's `run` method. If the workload cannot be statically scheduled across places, users need to provide an `initialize` method so that GLB will initialize this root task at place 0.

## 2.4 GLB internals

GLB implements the lifeline work-stealing algorithm described in [23]. We summarize the algorithm here:

1. Each Worker repeatedly calls `process(n)` method until it is running out of work. Between each process(n) call, Worker probes the network and responds to the stealing requests from other Workers.

2. When running out of work, a Worker conducts two rounds of work-stealing. It first chooses at most $w$ random victims. If none of the random victims has work to share, then Worker goes to the second round of work-stealing by stealing from its life-line buddies/victims. The topology of life-line communication graph is a $z$-dimensional hypercube. Such a topology satisfies these properties: it is a fully connected directed graph (so work can flow from any vertex to any other vertex) and it has a low diameter (so latency for work distribution is low) and has a low degree (so the number of buddies potentially sending work to a dead vertex is low). If a lifeline buddy cannot satisfy the stealing request due to its lack of work, it will still remember the request and try to satisfy the request when it gets work from others.

3. When all Workers run out of work, GLB terminates and applies the reduce function to each Worker's result to yield the final result.

Users can tune the GLB performance by changing these parameters: $w$, $z$, and $n$. It is more likely to steal work from a random victim with a larger $w$. It is more likely to steal work from a lifeline buddy when $z$ is larger. However, when $w$ and $z$ get larger, each worker spends more time probing the network and less time in processing tasks. Alternatively, users can make a Worker spend more time on processing tasks by increasing $n$. The larger $n$ is, the more tasks a Worker needs to finish before it can respond to the stealing requests. However, a large $n$ can sometimes hurt the performance if stealing requests need be responded fast. We will see an extreme case in which even $n = 1$ is too large a task-granularity in Section 2.6.

To help users understand and tune GLB program performance, GLB also provides logging functionalities to record (1) how much time each Worker spent on processing and distributing work (2) how many (random/lifeline) stealing requests each Worker sent and received. (3) how many (random/lifeline) stealings each Worker perpetrated. (4) How much workload has each Worker received/sent.

To better illustrate how to use GLB, we show a pedagogical use case via the Fibonacci problem in the Appendix.

## 2.5 UTS

### 2.5.1 Problem statement

The Unbalanced Tree Search benchmark measures the rate of traversal of a tree generated on the fly using a splittable random number generator. For this submission, we used a *fixed geometric law* for the random number generator with branching factor $b_0 = 4$, seed $r = 19$, and tree depth $d$ varying from 13 to 20 depending on core counts and architecture.

> The nodes in a geometric tree have a branching factor that follows a geometric distribution with an expected value that is specified by the parameter $b_0 > 1$. The parameter $d$ specifies its maximum depth cut-off, beyond which the tree is not allowed to grow ... The expected size of these trees is $(b_0)^d$, but since the geometric distribution has a long tail, some nodes will have significantly more than $b_0$ children, yielding unbalanced trees.

The depth cut-off makes it possible to estimate the size of the trees and shoot for a target execution time. To be fair, the distance from the root to a particular node is never used in our benchmark implementation to predict the size of a subtree. In other words, all nodes are treated equally, irrespective of the current depth.

Clearly, UTS is a case that static load-balancing does not work. Assume the sequential implementation of UTS is available (i.e., the code to grow the UTS tree and count the nodes), we now discuss how to apply GLB to it.

### 2.5.2 UTS TaskBag and TaskQueue

**UTS TaskBag** The internal representation of a UTS tree node is a triple(descriptor, low, high). Descriptor represents the hashed value of the node, low represents the smallest index of un-explored children, high represents the largest index of its un-explored children. The representation of a UTS tree is thus an array of UTS tree nodes. A UTS TaskBag is essentially a UTS tree. To split a UTS TaskBag, we evenly split each UTS node $n(d, l, h)$ to two nodes $n1(d, l, h1)$ and $n2(d, h2, h)$, where $h1$ and $h2$ are the middle points of $(l, h)$. If none of the UTS tree node has more than one child node, then we do not split the tree, as it is cheaper to count the node locally than move it to a remote place and count it there. To merge a UTS TaskBag, we simply concatenate the incoming TaskBag's UTS node array to the local one.

**UTS TaskQueue** `process(n)` method counts at most $n$ UTS tree nodes. `reduce()` is a straightforward sum-reduction method on each place's UTS tree node count.

Finally, GLB initializes the workload by initializing the root UTS tree node at Place 0.

## 2.6 BC

### 2.6.1 Problem statement

The Betweenness Centrality benchmark is taken from the SSCA2 (Scalable Synthetic Compact Application 2) v2.2 benchmark [1]; specifically, we implement the "fourth" kernel in this benchmark, the computation of *betweenness centrality*:

> The intent of this kernel is to identify the set of vertices in the graph with the highest betweenness centrality score. Betweenness Centrality is a shortest paths enumeration-based centrality metric, introduced by Freeman (1977). This is done using a betweenness centrality algorithm that computes this metric for every vertex in the graph. Let $\sigma_{st}$ denote the number of shortest paths between vertices $s$ and $t$, and $\sigma_{st}(v)$ the number of those paths passing through

$v$. Betweenness Centrality of a vertex $v$ is defined as $BC(v) = \Sigma_{s \neq v \neq t \in V} \sigma_{st}(v)/\sigma_{st}$.

The output of this kernel is a betweenness centrality score for each vertex in the graph and the set of vertices the highest betweenness centrality score.

We implement the Brandes' algorithm described in the benchmark, augmenting Dijkstra's single-source shortest paths (SSSP) algorithm, for unweighted graphs. We have implemented the exact variant of the benchmark (`K4approx=SCALE`).

The solution we implement makes one very strong assumption: that the graph is "small" enough to fit in the memory of a single place. Since the graph itself is not modified during the execution of this benchmark, it thus becomes possible to implement this benchmark by replicating the graph across all places. Now effectively the computation can be performed in an embarrassingly parallel fashion. The set of $N$ vertices is statically partitioned among $P$ places; each place is responsible for performing the computation for the *source* vertices assigned to it (for all $N$ target vertices) and computes its local `betweennessMap`. After this is done an `allReduce` collective performs an `AND` summation across all local `betweennessMaps`.

Clearly, statically partitioning the work amongst all places in this way is possible but not ideal. The amount of work associated with one source node $v$ vs another $w$ could be dramatically different. Consider for instance a degenerate case: a graph of $N$ vertices, labeled $1 \ldots N$, with an edge $(i, j)$ if $i < j$. Clearly the work associated with vertex 1 is much more than the work associated with vertex $N$.

We next discuss how to dynamically load-balance these tasks across all places using the GLB framework.

#### 2.6.2 BC TaskBag and TaskQueue

**TaskBag** Each vertex interval is a task item. We use a tuple(low, high) to represent a vertex interval. Each task bag is an array of such tuples. To split a TaskBag, we divide each tuple evenly. To merge a BC taskbag, we simply concatenate the incoming TaskBag's array to the local one.

**TaskQueue** `process(n)` method iterates over the taskbag and calculates the first n vertices. **reduce()** method is a simple betweenness-map (a vector) element-wise add function.

This implementation achieves linear speedup and perfect scalability on small-scale machines (i.e., 256 cores on Power) for smaller graph. However, it does not yield equally impressive performance on larger machines for larger graphs. After examining the GLB log, we realized that on large scale machines, Workers are less responsive to the stealing requests thus workload cannot be distributed fast. Therefore, we tried to maximize $w$ and $z$ and minimize $n$. (Please refer to Section 2.4 for the rationale of turning these parameters). The performance only improved slightly. We then realized that it took a Worker too long before it responded to the work stealing requests even when its task granularity is **one** vertex. So we changed the code that computes each vertex to an interruptable state machine. In this way, a Worker can respond to stealing requests without completing one vertex computation. Alternatively, we can help users to minimize the code change by providing a yielding functionality in the GLB library. Users can insert yield points in the their code to increase its probing frequency and responsiveness to stealing requests. We plan to provide such functionality in GLB in the future work.

## 3 Performance evaluation

### 3.1 Methodology

### 3.2 Unified code base

We compare our GLB code to the legacy X10 implementation used to evalutate X10 performance at Peta-scale [25]. To ensure a fair comparison, we use the same piece of sequential computation code for the legacy code and GLB code.

### 3.3 Platforms

We evaluated the performance on 3 different architectures: Power 775, Blue Gene/Q, and K. We now briefly discuss each architecture and the compiler options we used.

**Power-775** We gathered performance results on a small Power 775 systems with two drawers. A Power 775 octant (or host or compute node) is composed of a quad-chip module containing four eight-core 3.84 Ghz Power7 processors, one optical connect controller chip (codenamed *Torrent*), and 128 GB of memory (in our current configuration). A single octant has a peak performance of 982 GFLOPS; a peak memory bandwidth of 512 GB/s; and a peak bi-directional interconnect bandwidth of 192 GB/s. operating system image. Eight octants are grouped together into a drawer.

Each octant runs RedHat Enterprise Linux 6.2 and uses IBM's PE MPI for network communication (over PAMI).

We compiled the benchmark programs using Native X10 version 2.4.0 with `-NO_CHECKS`, `-O` options, and compiled the resulting C++ files with xlC 12.1 with the `-qinline -qhot -O3 -q64 -qarch=auto -qtune=auto` compiler options.

We allocated 32 places per octant when we use multiple octants. We used regular 64 KB pages for all the programs.

**Blue Gene/Q** Our Blue Gene/Q numbers were gathered on Vesta, a small Blue Gene/Q system located at Argonne National Laboratory. Each compute node of the Blue Gene/Q is a 64-bit PowerPC A2 processor with 16 1.6 Ghz compute cores and 16 GB of DRAM. The compute notes are grouped into drawers (32 compute nodes per drawer), which are grouped into midplanes (16 compute drawers, 512 compute nodes). Within the midplane, nodes are electrically connected in a 5-D torus (4x4x4x4x2). Beyond the midplane, an optical interconnect is used.

All benchmarks were compiled using Native X10 version 2.4.0 with `-NO_CHECKS`, `-O` options. The generated C++ code was then compiled for Blue Gene/Q using xlC v11.1 with the `-O3 -qinline -qhot -qtune=qp -qsimd=auto -qarch=qp` command line arguments.

To maximize the number of X10 Places, all experiments with 16 Places or more were run using the `c16` mode, which creates one MPI process (i.e., one X10 Place) per Blue Gene/Q compute core. Thus each X10 Place has 1 A2 core and 1 GB of DRAM available to it.

**K** The K computer is a supercomputer at RIKEN Advanced Institute for Computational Science. It consists of 82944 compute nodes, and each node has one scalar CPU (SPARC64™ VIIIfx, 8 cores, 128 GFLOPS) and 16 GB memory. These compute nodes are connected by Tofu interconnect,

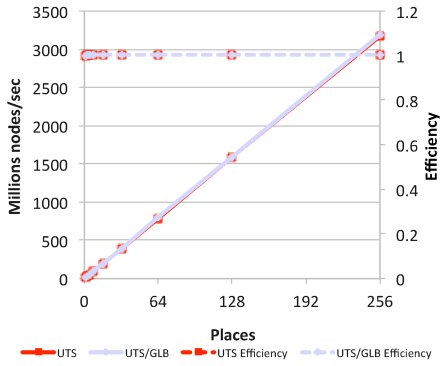


Figure 2: UTS/UTS-G Performance Comparison (on Power 775)

which is 6D mesh/torus network having 10 links (5 GB/s x 2 bandwidth per link) on each node.

Each node runs Linux-based OS, and developers can use MPI-2.1 for network communication.

We compiled the benchmark programs using Native X10 version 2.4.0 with `-NO_CHECKS`, `-O`, and `-FLATTEN_EXPRESSIONS` options, and compiled the resulting C++ files with Fujitsu C/C++ Compiler version K-1.2.0-14 with `-Xg` and `-Kfast` options.

We allocated 8 places per node when we use multiple compute nodes. The memory page sizes were set as 32 MB for all the programs.

### 3.4 Experimental results

### 3.5 UTS

In this section, we demonstrate the UTS-GLB performance by showing three figures, one per each architecture. On each architecture, we compare the legacy code to the GLB implementation. One thing to note is that the legacy code is a highly tuned parallel implementation of UTS and it won the HPCC2012 performance award [25]. On each figure, the x-axis represents the number of places; the primary y-axis (the left y-axis) represents the number of UTS nodes counted per second; the secondary y-axis (the right y-axis) represents the efficiency, which is calculated by how many nodes counted per second per place. A straight line whose slope is 1 along the primary y-axis indicates a linear speedup and a horizontal line on the secondary y-axis indicates the implementation scales perfectly. In the following discussion, we refer to UTS legacy code by UTS and refer to UTS GLB code by UTS-G.

Figure 2 shows the performance comparison for UTS and UTS-G on Power 775 up to 256 cores. UTS and UTS-G both achieve linear speedup and their efficiencies stay at 1, which means they both achieve perfect scaling.

Figure 3 shows the performance comparison for UTS and UTS-G on Blue Gene/Q up to 16384 cores. UTS and UTS-G both achieve linear speedup and perfect scaling.

Figure 4 shows the performance comparison for UTS and UTS-G on Blue Gene/Q up to 8192 cores. UTS and UTS-G perform well up to 4096 cores (i.e., almost linear speedup and efficiency stays above 0.9). When running on more than 4096 cores, both UTS and UTS-G drop their efficienies to 0.6. We are still investigating why there is a sudden performance drop on K.

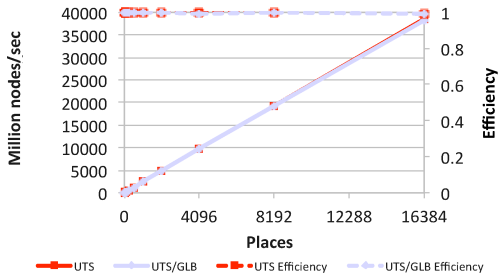


Figure 3: UTS/UTS-G Performance Comparison (on Blue Gene/Q)

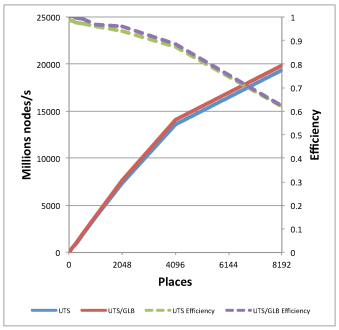


Figure 4: UTS/UTS-G Performance Comparison (on K)

**Summary** UTS-G achieves near linear speedup and perfect scaling on both Power 775 (up to 256 cores) and Blue Gene/Q (up to 16384 cores). On all three architectures, UTS-G achieves similar (or better) performance compared to UTS, a highly tuned award-winning implementation.

### 3.6 BC

In this section, we compare the BC-GLB performance to the BC legacy code. Two things to note are: (1) the legacy BC code uses a very different synchronization scheme from the legacy UTS code. In comparison, the GLB implementation of UTS and BC share exactly the same synchronization scheme (i.e., the underlying GLB library). This demonstrates GLB is applicable to widely different types of problems. (2) The legacy BC implementation randomizes which vertices to compute on each place, which effectively reduces the imbalance among places. As the number of places increases, such imbalance decreases. When running on more than 1024 places, BC's performance is only roughly 15% worse than then optimal performance (i.e., the longest execution time per place is only 15% more than the average finishing time across all places). Therefore, it is difficult to improve the performance because the room to improve is limited.

We demonstrate the UTS-GLB performance by showing six figures, two per each architecture. On each architecture, we first compare performance, then we compare the workload distribution among places. On each performance comparison figure, the x-axis represents the number of places; the primary y-axis (i.e., the left y-axis) represents the number of edges traversed per second; the secondary y-axis (i.e., the right y-axis) represents its efficiency, which is calculated by number of edges traversed per second per place. A straight line whose slope is 1 along the

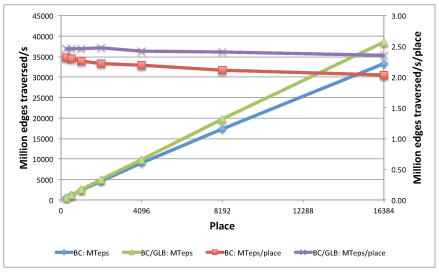


Figure 5: BC/BC-G Performance (on Blue Gene/Q)

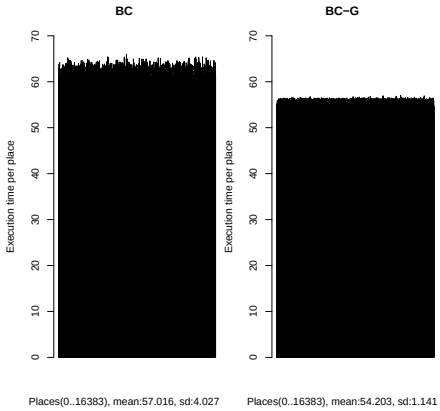


Figure 6: BC/BC-G Workload Distribution (on Blue Gene/Q)

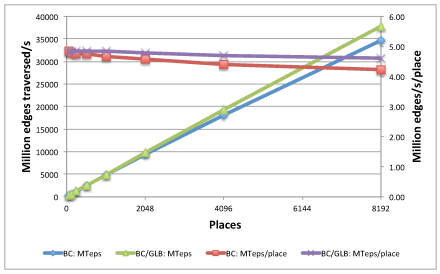


Figure 7: BC/BC-G Performance (on K)

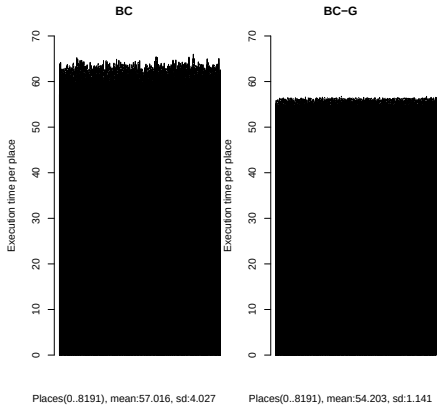


Figure 8: BC/BC-G Workload Distribution (on K)

primary y-axis indicates a linear speedup and a horizontal line along the secondary y-axis indicates the implementation scales perfectly. To understand how effective GLB is at balancing the workload compared to the legacy code, we also plot the workload distribution graphs. On each workload distribution graph, we bar-plot the calculation time on each place and bundle them together. The more even workload distribution graph appears, the more balanced workload is. A rectangular visualization indicates the perfect load-balancing. We also show the mean and standard deviation of the workload distribution on each figure. In the following discussion, we refer to BC legacy code as BC and refer to BC-GLB code as BC-G.

Figure 5 shows the BC performance comparison on Blue Gene/Q. We can see that GLB implementation achieves near linear speedup and near perfect scaling.

Figure 6 shows the workload distribution of BC on Blue Gene/Q. As we can see, BC-G workload distribution is much more even than that of BC, GLB reduces the standard deviation from 4.027 to 1.141. In fact, BC-G finishes calculation in 57.0586 seconds, which is almost equal to the mean of BC computing time accross all places (57.015s). This indicates that BC-G achieves the near perfect load-balancing on Blue Gene/Q.

Figure 7 shows the BC performance comparison on K. BC-G also achieves near linear speedup and near perfect scaling (efficiency>0.95 up to 8192 places).

Figure 8 shows the workload distribution of BC on Blue Gene/Q. As we can see, BC-G workload distribution is much more even than that of BC on K. BC-G finishes calculation in 58.198729 seconds on 8192 places, which is within 1.5% of the mean of BC computing time across all places (57.016s). This indicates that BC-G also achieves the near perfect load-balancing on K.

Figure 9 shows the BC performance comparison on K. Surprisingly, BC-G on Power 775 does not achieve the linear speedup and perfect scaling as it does on Blue Gene/Q and on K. That is because as the number of places increase, the total amount of time to finish calculating all the vertices also increase even when the graph stays the same. Additionally, for the same workload, BC-G spends more time calculating (on average 5-20% more) than BC when both run on a single place. We suspect both problems are due to the overly sensitive C++ compiler optimizations on Power 755; we are still investigating to verify this hunch. However, as demonstrated in Figure 10, BC-G is very effective at removing the imbalance among workloads, it decreases the workload standard deviation from 58.463 to 1.482.

**Summary** BC-G constantly outperforms the BC implementation. On Blue Gene/Q (up to 16384 cores) and K (up to 8192 cores), BC-G achieves the near-optimal speedup, scalability and load-balancing.

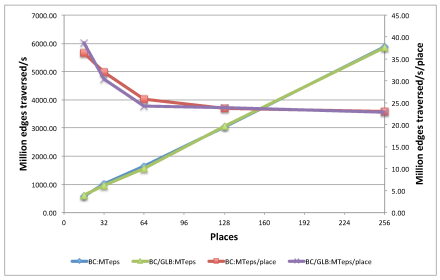


Figure 9: BC/BC-G Performance (on Power 775)

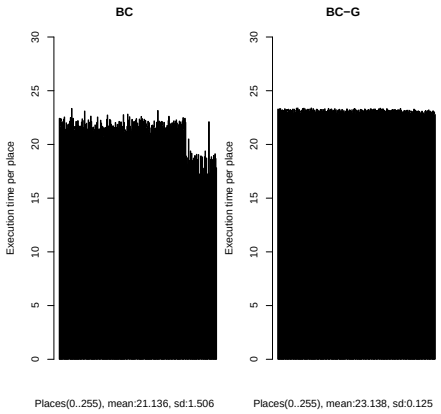


Figure 10: BC/BC-G Workload Distribution (on Power 775)

## 4 Future work

We plan to do the following future work: (1) Investigate the performance anomaly of UTS-G on K and BC-G on Power 755. (2) Provide yield points in the GLB library so that users can minimize the changes to the existing sequential code and improve the GLB program's responsiveness to work stealing requests. (3) Experiment with more benchmarks. (4) Provide a mechanism to auto-tune GLB parameters (e.g., task granularity, size of random victims/lifeline buddies).

## 5 Related work

There is extensive prior work on dynamic load balancing for shared-memory systems. The Cilk [9] system was the first to provide efficient load balancing for a wide variety of irregular applications. Load balancing in Cilk applications is achieved by a scheduler that follows the *depth-first work, breadth-first steal* principle [4]. Cilk's scheduling strategy, in which each worker maintains its own set of tasks and steals from other workers when it has nothing to do, is often referred to as *work-stealing.* Following Cilk, many runtimes for shared memory task parallelism utilize work-stealing schedulers. Of these, OpenMP 3.0 [17], Intel's Threading Building Blocks (TBB) [19], Java Fork-Join [14], Microsoft's Parallel Patterns Library (PPL), and Task Parallel Library (TPL) are the most popular. utilized work-stealing schedulers including X10's *breadth-first* [6] and *compiler-supported* [26] and Guo et al.'s [11] hybrid model for work stealing.

The techniques for dynamic load balancing in shared-memory environments are not directly applicable to distributed-memory machines because of a variety of issues such as network latency and bandwidth, and termination detection. Various bodies of work have addressed the problem of dynamic load balancing on distributed-memory machines. Grama et al. [13, 10] discuss various load balancing strategies for distributed parallel searches that are independent of the specific search technique. Blumofe et al. [5] adapt the Cilk work-stealing model to distributed shared memory by limiting the scope of the programs to be purely functional. ATLAS [2] and Satin [27] both use hierarchical work-steal to acheive global load balancing. Charm++ [24, 12] monitors the execution of its distributed programs for load imbalance and migrates computation objects to low-load places to correct the load imbalance. Global load balancing for message passing environments has been researched for specific problems by Batoukov and Sorevik [3].

UTS, an excellent example of an irregular application, was first described by Prins et al. [18]. It has since been widely used as a benchmark for dynamic load balancing. Olivier and Prins [16] provided the first scalable implementation of UTS on clusters that provided up to 80% efficiency on 1024 nodes. To this end, they employed a custom application-level load balancer along with optimizations such as one-sided communications and novel termination detection techniques. Dinan et al. [8] provide a framework for global load balancing, which was used to achieve speedups on UTS on 8196 processors. Global load balancing and termination detection facilities were provided to express irregular applications. By reserving one core per compute node on the cluster exclusively for lock and unlock operations, this framework allowed threads to steal work asynchronously without disrupting the victim threads. However, the cost paid was a static allocation (one core out of every eight) for communication. This results in lower throughput because the thread is not available for user-level computations. Saraswat et al. [23] introduced lifeline-based global load balancing and showed 87% efficiency on 2048 nodes. Later work by the X10 team demonstrated 98% parallel efficiency with 55,680 Power7 cores [25]. An implementation of the life-line algorithm in Co-Array Fortran achieved 58% efficiency at 8192 nodes [15]. A more recent UTS code using CAF 2.0 `finish` construct achieves a 74% parallel efficiency on 32,768 Jaguar cores [28].

## Acknowledgments

The code for BC was developed in collaboration with Anju Kambadur. Earlier versions of the code presented here were worked on by Sreedhar Kodali, Ganesh Bikshandi, David Cunningham, Pradeep Varma, Krishna Venkat and other colleagues.

This material is based upon work supported by the Defense Advanced Research Projects Agency under its Agreement No. HR0011-07-9-0002, by the Department of Energy and by the Air Force Office of Scientific Research.

Some of the results were obtained by using the K computer at the RIKEN Advanced Institute for Computational Science. This research was partially supported by JST, CREST.

This research used resources of the Argonne Leadership Computing Facility at Argonne National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy under contract DE-AC02-06CH11357.

## References

[1] D. A. Bader, J. Feo, J. Gilbert, J. Kepner, D. Koester, E. Loh, K. Madduri, B. Mann, and T. Meuse. HPCS Scalable Synthetic Compact Applications #2: Graph Analysis. `http://www.graphanalysis.org/benchmark/HPCS-SSCA2_Graph-Theory_v2.2.pdf`, 2007.

[2] J. E. Baldeschwieler, R. D. Blumofe, and E. A. Brewer. Atlas: an infrastructure for global computing. In *EW 7: Proceedings of the 7th workshop on ACM SIGOPS European workshop*, pages 165–172, New York, NY, USA, 1996. ACM.

[3] R. Batoukov and T. Sorevik. A Generic Parallel Branch and Bound Environment on a Network of Workstations. In

*HiPer '99: Proceedings of High Performance Computing on Hewlett-Packard Systems*, pages 474–483, 1999.

[4] R. D. Blumofe and C. E. Leiserson. Scheduling Multithreaded Computations by Work Stealing. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 356–368, 1994.

[5] R. D. Blumofe and P. A. Lisiecki. Adaptive and reliable parallel computing on networks of workstations. In *ATEC '97: Proceedings of the annual conference on USENIX Annual Technical Conference*, pages 10–10, Berkeley, CA, USA, 1997. USENIX Association.

[6] G. Cong, S. Kodali, S. Krishnamoorthy, D. Lea, V. Saraswat, and T. Wen. Solving Large, Irregular Graph Problems Using Adaptive Work-Stealing. In *ICPP '08: Proceedings of the 2008 37th International Conference on Parallel Processing*, pages 536–545, Washington, DC, USA, 2008. IEEE Computer Society.

[7] J. Dean and S. Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Conference on Symposium on Opearting Systems Design & Implementation - Volume 6*, OSDI'04, pages 10–10, Berkeley, CA, USA, 2004. USENIX Association.

[8] J. Dinan, D. B. Larkins, P. Sadayappan, S. Krishnamoorthy, and J. Nieplocha. Scalable work stealing. In *SC '09: Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, pages 1–11, New York, NY, USA, 2009. ACM.

[9] M. Frigo, C. E. Leiserson, and K. H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the ACM SIGPLAN '98 Conference on Programming Language Design and Implementation*, pages 212–223, Montreal, Quebec, Canada, June 1998. Proceedings published in ACM SIGPLAN Notices, Vol. 33, No. 5, May, 1998.

[10] A. Grama and V. Kumar. State of the Art in Parallel Search Techniques for Discrete Optimization Problems. *IEEE Trans. on Knowl. and Data Eng.*, 11(1):28–35, 1999.

[11] Y. Guo, R. Barik, R. Raman, and V. Sarkar. Work-First and Help-First Scheduling Policies for Async-Finish Task Parallelism. In *Proceedings of the 23rd IEEE International Parallel and Distributed Processing Symposium*, May 2009.

[12] L. V. Kalé and S. Krishnan. CHARM++: A portable concurrent object oriented system based on C++. In *Proceedings of Object Oriented Programming Systems, Languages and Applications, ACM Sigplan Notes*, volume 28, pages 91–108, 1993.

[13] V. Kumar, A. Y. Grama, and N. R. Vempaty. Scalable load balancing techniques for parallel computers. *J. Parallel Distrib. Comput.*, 22(1):60–79, 1994.

[14] D. Lea. A Java fork/join framework. In *Proceedings of the ACM 2000 conference on Java Grande*, JAVA '00, pages 36–43, New York, NY, USA, 2000. ACM.

[15] J. Mellor-Crummey, L. Adhianto, G. Jin, M. Krentel, K. Murthy, W. Scherer, and C. Yang. Class II Submission to the HPC Challenge Award Competition Coarray Fortran 2.0, Nov. 2011.

[16] S. Olivier and J. Prins. Scalable dynamic load balancing using UPC. In *ICPP '08: Proceedings of the 2008 37th International Conference on Parallel Processing*, pages 123–131, Washington, DC, USA, 2008. IEEE Computer Society.

[17] OpenMP Architecture Review Board. *OpenMP Application Program Interface, v3.0*. May 2008.

[18] J. Prins, J. Huan, B. Pugh, C.-W. Tseng, and P. Sadayappan. UPC Implementation of an Unbalanced Tree Search Benchmark. Technical Report 03-034, University of North Carolina at Chapel Hill, October 2003.

[19] J. Reinders. *Intel Threading Building Blocks*. O'Reilly, 2007.

[20] V. Saraswat, G. Almasi, G. Bikshandi, C. Cascaval, D. Cunningham, D. Grove, S. Kodali, I. Peshansky, and O. Tardieu. The Asynchronous Partitioned Global Address Space Model. In *AMP'10: Proceedings of The First Workshop on Advances in Message Passing*, June 2010.

[21] V. Saraswat, B. Bloom, I. Peshansky, O. Tardieu, and D. Grove. The X10 reference manual, v2.4. Sept. 2013.

[22] V. Saraswat and R. Jagadeesan. Concurrent clustered programming. In *Concur'05*, pages 353–367, 2005.

[23] V. A. Saraswat, P. Kambadur, S. Kodali, D. Grove, and S. Krishnamoorthy. Lifeline-based global load balancing. In *Proceedings of the 16th ACM Symposium on Principles and Practice of Parallel Programming*, PPoPP '11, pages 201–212, New York, NY, USA, 2011. ACM.

[24] A. B. Sinha and L. V. Kalé. A load balancing strategy for prioritized execution of tasks. In *IIPS'93: Proceedings of International Parallel Processing Symposium*, pages 230–237, 1993.

[25] O. Tardieu, B. Herta, D. Cunningham, D. Grove, P. Kambadur, V. A. Saraswat, A. Shinnar, M. Takeuchi, and M. Vaziri. X10 and APGAS at Petascale. In *Proceedings of the 19th ACM Symposium on Principles and Practice of Parallel Programming*, PPoPP '14. ACM, 2014.

[26] O. Tardieu, H. Wang, and H. Lin. A work-stealing scheduler for X10's task parallelism with suspension. In *Proceedings of the 17th ACM SIGPLAN symposium on Principles and Practice of Parallel Programming*, PPoPP '12, pages 267–276, New York, NY, USA, 2012. ACM.

[27] R. V. van Nieuwpoort, T. Kielmann, and H. E. Bal. Efficient load balancing for wide-area divide-and-conquer applications. In *PPoPP '01: Proceedings of the eighth ACM SIGPLAN symposium on Principles and practices of parallel programming*, pages 34–43, New York, NY, USA, 2001. ACM.

[28] C. Yang, K. Murthy, and J. Mellor-Crummey. Managing asynchronous operations in coarray fortran 2.0. In *IEEE 27th International Symposium on Parallel Distributed Processing (IPDPS)*, pages 1321–1332, 2013.

```
1   public class FibG(n:Long) {
2       class FibTQ implements TaskQueue{
3           val bag = new ArrayListTaskBag[Long]();
4           var result:Long=0;
5           public def getTaskBag()=bag;
6           public def getResult()= result;
7           public def init(n:Long) {
8               bag.bag().add(n);
9           }
10          public def process(var n:Long):Boolean {
11              val b = bag.bag();
12              for (var i:Long=0; bag.size() > 0 && i < n; i++) {
13                  val x = b.removeLast(); // constant time
14                  if (x < 2) result += x;
15                  else {
16                      b.add(x-1); // constant time
17                      b.add(x-2);
18                  }
19              }
20              return b.size()>0;
21          }
22          public def merge(var _tb:glb.TaskBag):void {
23              this.bag.merge(_tb as ArrayListTaskBag[Long]);
24          }
25          public def split():glb.TaskBag {
26              return this.bag.split();
27          }
28          public def reduce():void {
29              result= Team.WORLD.allreduce(result,Team.ADD);
30          }
31      }
32      public def run():Long {
33          val init = ()=>{ return new FibTQ(); };
34          val glb = new GLB[FibTQ](init, GLBParameters.Default, true);
35          val start = ()=>{ (glb.taskQueue()).init(n); };
36          glb.run(start);
37          PlaceGroup.WORLD.broadcastFlat(()=>{
38                  (glb.taskQueue()).reduce();
39              });
40          return glb.taskQueue().result;
41      }
42      public static def main(args:Rail[String]) {
43          val N = args.size < 1 ? 10 : Long.parseLong(args(0));
44          val result = new FibG(N).run();
45          Console.OUT.println("fib-glb(" + N + ")" + result);
46
47      }
48  }
```

Figure 11: GLB-Fibonacci example

# Appendices

We demonstrate how to use GLB via a Fibonacci example. We show the complete code in Figure 11. Specifically, We apply GLB to Fibonacci($n$) in the following way:

1. *TaskBag* We use the default GLB Arraylist-based Taskbag, as shown at line 3. A task is an integer $i$ whose Fibonacci number should be computed. One can split/merge the TaskBag by calling the `merge` and `split` methods in the default TaskBag, as shown in Line22 – Line 27.

2. *TaskQueue* Each TaskQueue keeps a *result* of Long type, as shown at line 4. To process each task item $i$, we first judge if it is less than 2, if so, add $i$ to *result*; if not, remove $i$ from the TaskBag and add $i-1$ and $i-2$ to the TaskBag. When its TaskBag becomes empty, TaskQueue's *result* is the Fibonacci number of $n$. The `process` method is shown in line 10 – line 21. The reduce function is a simple add function, as shown in line 28 – line 30.

3. *Invoke GLB* Since it is difficult to statically schedule tasks among places for the Fibonacci problem, users can invoke GLB by calling the `run` method with an `initialize` method to use at the root place, as shown in line 35 and line 36. Were it easy to statically load balance, users could call the `run` method without providing the initialization method.

Note that only these three pieces of code are required from the users and users are oblivious to any synchronization mechanism in X10.